Agent Traces with MLFLow and Red Hat OpenShift AI

Version 1, 2026-06-19

Home

Goal

This course teaches how to deploy MLFlow, as included with Red Hat OpenShift AI 3.4; How to collect traces of an agentic application, using the MLFlow SDK with Python; and how to visualize those traces using the Red Hat OpenShift AI web interface.

This course is intended to require learners between 4 hours to one working day to complete.

Work In Progress

Objectives

On completing this course, you should be able to:

- Understand how MLFlow relates to agentic applications and to Red Hat OpenShift AI.
- Deploy a development environment with MLFlow
- Collect traces from an agentic application using MLFlow

Audience

- Software developers working on agentic applications and agentic harnesses.
- Red Hat and Partner roles that perform demonstrations, PoC and PoV of MLFlow with Red Hat OpenShift AI, such as Solutions Architects.
- Red Hat and Partner roles that implement and support MLFlow with Red Hat OpenShift AI, such as Services Consultants, Technical Account Managers, and Customer Support Engineers.

Prerequisites

This course assumes that you have the following experience:

- Familiarity with generative AI and agentic application concepts.
- Familiarity with the basic operation of Red Hat OpenShift AI.
- Python programming skills are recommended but not required, because this training provides sample code which is ready to run.

Classroom environment

The demo catalog entry Red Hat OpenShift AI 3.4 provides an SNO OpenShift cluster, backed by an AWS instance with a single GPU, with a predeployed llama-32-3b-instruct model which is capable of making tool calls.

This course uses that demo entry as-is, and any additional components which are required by

either MLFlow itself or any of the course activities will be configured by learners, as part of course activities, by using sample Python code and Kubernetes manifests provided in a public Git repository.

Other sources of information

For documentation about MLFlow with Red Hat OpenShift AI, see the Red Hat OpenShift AI product documentation.

For upstream MLFlow documentation, see the MLFlow project documentation.

Author

Fernando Lozano
Training Content Architect
Red Hat - Product Portfolio Marketing & Learning

Lab Environment

Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. **Log in** to the RHDP portal
2. **Search** for the catalog entry: **FIXME Lab Catalog Name**.
3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.
6. **Review the warning** at the bottom of the form and **check the box** labeled:
   “I confirm that I understand the above warnings.”
7. **Click** the **Order** button to place your lab order.

Important Notes:

- This lab may take approximately **FIXME minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

Understanding MLFlow and Red Hat OpenShift AI

Goal

Understand how MLFlow relates to agentic applications and to Red Hat OpenShift AI.

This chapter introduces MLFlow in the context of Red Hat OpenShift AI, focusing on features relevant to generative AI and agentic applications. You'll learn about MLFlow's architecture, supported features, and how it integrates with RHOAI components.

Introduction to MLFlow in Red Hat OpenShift AI

*Estimated reading time: **5 minutes**.*

Objective

Understand MLFlow's features for agentic applications and how it integrates with Red Hat OpenShift AI.

Not started yet.

MLFlow Features in RHOAI

Lorem ipsum

Focus on Generative AI Applications

Lorem ipsum

MLFlow Experiments

Lorem ipsum

What's Next

Now that you understand MLFlow's role in RHOAI and agentic applications, the next section provides a quiz to verify your understanding of key MLFlow concepts.

Quiz: MLFlow Concepts Review

*Estimated reading time: **5 minutes**.*

Objective

Verify your understanding of MLFlow concepts and terminology.

Interactive Assessment

What’s next

Now that you’ve verified your understanding of MLFlow concepts, the next chapter explores how to deploy MLFlow in your RHOAI environment for development purposes.

Deploying MLFlow for Development

Goal

Deploy a development environment with MLFlow

This chapter guides you through deploying MLFlow in Red Hat OpenShift AI for development purposes. You'll learn about MLFlow's architecture, how it integrates with RHOAI, and gain hands-on experience deploying and verifying your MLFlow instance.

MLFlow Architecture and RHOAI Integration

*Estimated reading time: **5 minutes**.*

Objective

Understand MLFlow's architecture and how it integrates with Red Hat OpenShift AI.

Not started yet.

MLFlow Components

Lorem ipsum

RHOAI Integration

Lorem ipsum

Development vs Production Deployments

Lorem ipsum

What's Next

Now that you understand MLFlow's architecture and integration with RHOAI, the next section provides hands-on experience deploying MLFlow in a development environment.

Lab: Deploy MLFlow on Red Hat OpenShift AI

*Estimated reading time: **5 minutes**.*

Objective

Deploy and verify a development MLFlow instance in Red Hat OpenShift AI.

Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Explore the demo environment and verify RHOAI features.
    a. Something to do, using a graphical or web UI. Click menu:Save As[] to save your work.
    b. More things to do
2. Enable MLFlow by editing the DSC custom resource.
    a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
highlight something in the output
```

    b. More things to do. Press kbd:[Ctrl+C] to interrupt.

Summary of what you accomplished.

What's next

Now that you have a working MLFlow deployment, the next chapter explores how to collect traces from agentic applications using the MLFlow SDK.

Collecting Traces from Agentic Applications

Goal

Collect traces from an agentic application using MLFlow

This chapter teaches you how to use the MLFlow SDK to collect traces from agentic applications. You'll learn why tracing is essential for troubleshooting agents, how to organize traces in experiments and runs, and gain hands-on experience adding MLFlow tracing to your code.

Tracing Agentic Applications

*Estimated reading time: **5 minutes**.*

Objective

Understand why tracing is essential for agentic applications and how MLFlow organizes trace data.

Not started yet.

Why Developers Need Traces

Lorem ipsum

MLFlow SDK and Trace Calls

Lorem ipsum

Experiments and Runs

Lorem ipsum

Significant Events in Traces

Lorem ipsum

What's Next

Now that you understand tracing concepts, the next section provides hands-on experience modifying an agent to record traces using the MLFlow SDK.

Lab: Modify an Agent to Record Traces

*Estimated reading time: **5 minutes**.*

Objective

Add MLFlow tracing to an agentic application and visualize the generated traces.

Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Prototype an agent using the GenAI playground.
    a. Something to do, using a graphical or web UI. Click menu:Save As[] to save your work.
    b. More things to do
2. Generate code and add MLFlow tracing calls.
    a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
highlight something in the output
```

    b. More things to do. Press kbd:[Ctrl+C] to interrupt.

Summary of what you accomplished.

What's next

Now that you've added basic tracing to an agent, the next section explores how to trace MCP tool calls to capture more detailed information about agent behavior.

Lab: Trace MCP Tool Calls from an Agent

*Estimated reading time: **5 minutes**.*

Objective

Deploy an MCP server and capture its tool invocations in MLFlow traces.

Not started yet.

Before you begin

Describe requirements, such as customer portal accounts, Red Hat Developer free sub, already configured machines, access to container registries, etc.

Instructions

High-level summary of what you will perform in this lab, but more detailed than the objective.

1. Deploy an MCP server.
    a. Something to do, using a graphical or web UI. Click menu:Save As[] to save your work.
    b. More things to do
2. Add the MCP server to your agentic application.
    a. Something to do, using a shell.

```
$ command --options fixed-arguments variable-arguments
output of the command
highlight something in the output
```

    b. More things to do. Press kbd:[Ctrl+C] to interrupt.

Summary of what you accomplished.

What's next

Congratulations! You've completed this course and learned how to deploy MLFlow, collect traces from agentic applications, and visualize trace data. You can now apply these skills to troubleshoot and improve your own agentic applications.